

 Audit_1.pdf
7.79 MB

 binance_broker.py
34.09 KB

 Engine_1.py
166.19 KB

 live_unified_pre...
34.92 KB

 run_all_6.py
10.53 KB

 six_strategy_en...
29.79 KB

 train_six_strateg...
18.08 KB

Context

Across initial development, backtesting, and live testing (Part 1, Part 2, and Part 3), we uncovered 28 critical vulnerabilities across five core architectural layers. Below is the complete catalog of defects, failure modes, root causes, and architectural lessons for deep-dive peer review with Codex.

Layer 1: Scraper DOM & Browser Automation Defects

1. **"The Self-Locking Zero-Guard Trap ("Stuck Values")****
 - "Failure Mode": Scraped indicator values froze permanently after initial seeding while the terminal continued reporting a healthy 2.0 FPS.
 - "Root Cause": When TradingView DO legend elements were unrendered during a poll cycle, 'SINGLE_FRAME_EXTRACTION.js' returned "N/A", which "parse_float" coerced to "0.0".
 - "SnapshotStore update()" had a protective zero-guard ("if v == 0.0 and cur_val != 0.0: continue") to prevent 0.0 overwriting HTTP interrupts. Once an indicator had a valid non-zero value, any subsequent "N/A" cycle was permanently rejected, locking the value indefinitely.
 - "Fix Applied": Filtered out "N/A" fields at the JS/Python boundary before dispatching to "update()", passing only genuinely extracted numbers.
2. **"Hardcoded Telemetry Masking Silent Scrapor Outages****
 - "Failure Mode": Scrapor stream panel showed "F2.0 | Age: 0s" even when 8 of 9 chart iframes were broken.
 - "Root Cause": "scrapor_fps" was hardcoded to "2.0" whenever any single iframe extraction returned "True".
 - "Fix Applied": Replaced static assignment with elapsed-time throughput counters and added an explicit 'Frames: X/9' success ratio to the health dashboard.
3. **"Watchdog Blindness via Price-Stream Contamination****
 - "Failure Mode": Auto-healing watchdog never reloaded broken Playwright tabs when | ↓ indicators died.
 - "Root Cause": Watchdog evaluated 'last_change_ns', which was updated on every incoming tick from the independent Binance WebSocket and REST price feeds. Because price was updating, the watchdog considered the entire tab was healthy.
 - "Fix Applied": Isolated 'scrapal_val_ns' to track indicator fields exclusively, triggering watchdog auto-reload if 50% or more symbols show no indicator updates for 120 seconds.
4. **"DOM Cursor Hover Price Poisoning**"**
 - "Failure Mode": Live trades surfaced immediate false exits and stop-loss triggers.
 - "Root Cause": Moving the mouse over historical candles in the TradingView chart updated the DOM legend OHLC. The scrapor extracted this historical price as 'price_val', overwriting the live market price in 'SnapshotStore'.
 - "Fix Applied": Blocked all price/OHLC updates from 'source=coinglass', restricting price ingestion exclusively to Binance WebSocket and REST feeds.
5. **"Multi-Window Seeding Keyboard Collision**"**
 - "Failure Mode": Ticker symbols in chart search boxes became scrambled (e.g., "BTTCRXUSD"), resulting in invalid symbols across iframes.
 - "Root Cause": Concurrent asynchronous tasks typing ticker symbols into 9 search boxes simultaneously shared focus on the same active page.
 - "Fix Applied": Serialized seeding sequentially per tab and introduced isolated browser profile directories ("chrome_profile_tab1" on port 9222 and "chrome_profile_tab2" on port 9223).
6. **"Playwright Context Detachment on Persistent Chromium**"**
 - "Failure Mode": Scrapor threw "TargetClosedError" during initialization.
 - "Root Cause": Calling "context.new_page()" on persistent contexts left the default "pages[0]" unmanaged, triggering detachment during layout transitions.
 - "Fix Applied": Initializ↓ed page handles strictly via "context.pages[0]" if context pages else await context.new_page()".
7. **"TradingView Layout Cloud Sync State Bleed**"**
 - "Failure Mode": Injected indicators in Tab 1 spontaneously mirrored into Tab 2 charts, overwriting commodity indicator configurations.
 - "Root Cause": TradingView layout 's9' autosaved chart states to the cloud session.
 - "Fix Applied": Hardcoded "autoSaveEnabled = false" across all chart iframe sessions.
8. **"Track Count Indicator Period Regex Shift**"**
 - "Failure Mode": Take/buy/sell volume counts displayed "14" and "520" instead of true transaction counts.
 - "Root Cause": When the indicator legend header returned three numbers [14, 520, 480], "numVals[numVals.length - 3]" extracted "14" (the indicator's lookback period) instead of "buy count".
 - "Fix Applied": Added specific structural regex parsing keyed on header label text rather than raw positional slices.
9. **"Single Aggregated Liquidation DOM Truncation**"**
 - "Failure Mode": "liq_short" remained "0.0" indefinitely on live crypto symbols.
 - "Root Cause": If the UI collapsed liquidation display into a single aggregate metric, the JS parser only populated "liquidations_long".
 - "Fix Applied": Added HTTP interrupt fallback on "liquidation" network payloads and fallback parsing rules for single-metric signed values.

Layer 2: Quantitative, Mathematical & Training Defects

10. ****VWAP Expanding Variance Formula Error****
- **"Failure Mode":** Mean-reversion bands (2.2 and 3.3 std dev) collapsed unnaturally tight again. ↓ JAP, causing premature false entries.
 - **"Root Cause":** 'live_model_trainer.py' computed variance by subtracting the past bar's VWAP inside 'cumsum()':
`'(((typical_price - vwap) ** 2) * volume).groupby(session_group).cumsum() / cum_v'`
 - **"Fix Applied":** Replaced with the mathematically sound expanding sample variance formula:
`'sigma^2 = (sum(volume * Price^2) / sum(volume)) - (VWAP_k)^2'`.
11. ****BTC Reference Join Cross-Market Misalignment****
- **"Failure Mode":** Altcoin trend features ('z_btc_4', 'z_btc_10', 'z_btc_20') were corrupted with data from unrelated market regimes.
 - **"Root Cause":** 'btc_ref' lacked a timestamp column ('ts'), causing pandas to execute a positional index join. If BTC had 1,200 bars and the altcoin had 500 bars, row 0 of the altcoin matched row 0 of BTC from days earlier.
 - **"Fix Applied":** Enforced explicit datetime alignment on 'open_time' before merging.
12. ****Pre-Split Rolling Feature Leakage****
- **"Failure Mode":** Overstated model validation scores that collapsed in out-of-sample forward testing.
 - **"Root Cause":** ATR and CVD rolling z-scores were computed across the entire historical dataset before executing the 70/30 train/test split.
 - **"Fix Applied":** Calculated rolling feature statistics strictly within training folds, applying fitted transformations to out-of-sample segments.
13. ****RSI Division-by-Zero Bias****
- **"Failure Mode":** Extended flat consolidation bars produced an RSI of '0.0' instead of a neutral '50.0'.
 - **"Root Cause":** Division by '1e-10' on zero price gain/loss evaluated to 0.0.
 - **"Fix Applied":** Added zero-differential guards returning 50.0 when both gains and losses are zero.
14. ****ML Fail-Open Exception Handling****
- **"Failure Mode":** Model feature dimension mismatches allowed unvalidated raw signals to bypass ML probability thresholds.
 - **"Root Cause":** Inference execution wrapped in 'except Exception: pass' swallowed model failures silently.
 - **"Fix Applied":** Changed exception handler to fail-closed, blocking signal dispatch on any inference failure.
15. ****S1 Strategy Heuristic Filter Trivial No-Op****
- **"Failure Mode":** Strategy 1 bypassed heuristic pre-filtering, running expensive ML inference unconditionally on every tick.
 - **"Root Cause":** Heuristic check contained 'lim * 0.0', which evaluated to 0.0 on all inputs.
 - **"Fix Applied":** Corrected algebraic expression to evaluate intended liquidation magnitude thresholds.
-

Layer 3: Backtest (OOS) vs. Live Engine Parity Mismatches

16. ****TP Target Label Mismatch (The 7R vs 5R Selection Bias)****
- **"Failure Mode":** Live engine suffered continuous stop-outs while OOS models showed high win rates.
 - **"Root Cause":** 'live_model_trainer.py' trained models with 'TP_MULT_OPTIONS = [7.0]', labeling any trade reaching 4R to 6.9R as '0' (Loss). The live engine exited at 5.0R. The models were explicitly trained to reject the exact trade distributions the engine intended to capture.
 - **"Fix Applied":** Unified 'TP_MULT_OPTIONS' across trainer, simulator, and live engine to '5.0'.
17. ****Stop Loss Floor Override Distorting Risk-Reward Geometry****
- **"Failure Mode":** Asymmetric loss sizing and broken Sharpe ratios.
 - **"Root Cause":** Thr-'hardcoded' 'min_sl_pct = 0.003' overrides widened ATR stop loss. ↓ p to 3x on low-volatility assets (e.g. BTC at \$65k with \$68 ATR expanded to \$195 SL), distorting 5:1 R:R geometry.
 - **"Fix Applied":** Removed hardcoded floor overrides, preserving pure ATR stop geometry with strict minimum volatility filters.
18. ****Fractional Kelly Hyperparameter Selection vs. Live Flat Sizing****
- **"Failure Mode":** Hyperparameters selected in Optuna failed when run live.
 - **"Root Cause":** OOS backtests scored candidate parameters on 'adj_pnl = net_pnl * kelly_mult', selecting parameters that depended on variable sizing, while live execution operated on flat risk per trade.
 - **"Fix Applied":** Harmonized parameter optimization to evaluate on flat ATR-unit returns.
19. ****Execution Timing Discrepancy (Open[i+1] vs Close[i])****
- **"Failure Mode":** Live fills lagged backtest performance by 3 to 10 bps per trade.
 - **"Root Cause":** Backtest simulated entries on the subsequent bar open ('Open[i+1]'), whereas live engine submitted orders on confirmed bar close ('Close[i]').
 - **"Fix Applied":** Calibrated slippage models in backtesting to match confirmed close market orders.
20. ****Missing Live VWAP-Recross Exit for Strategies S4, S5, S6****
- **"Failure Mode":** Live mean-reversion trades gave back all accumulated profits and stopped out.
 - **"Root Cause":** OOS backtests exited mean-reversion trades the moment price touched VWAP. Live engine lacked a VWAP-touch exit listener and held positions through the recross until hitting trailing stops.
 - **"Fix Applied":** Implemented an intra-candle VWAP-cross exit trigger in 'LiveStrategyPredictor'.
21. ****EMA Warm-Up Period Discrepancy****
- **"Failure Mode":** Live trend strategies produced zero trades during the first 800 bars of startup ↓
 - **"Root Cause":** OOS used 'min_periods=50/100', while live engine defaulted to full span length (200/800 bars), producing 'NaN' macro regime scores.
 - **"Fix Applied":** Set 'min_periods=50' on live indicator rolling buffers.
22. ****Incomplete Candle Featurization on Unclosed Bars****
- **"Failure Mode":** High-frequency false signals fired 2 minutes into a 15-minute bar.
 - **"Root Cause":** ATR and delta indicators were evaluated against live in-progress bars rather than strictly confirmed closed candles.
 - **"Fix Applied":** Added a 'candle_is_closed' validation gate before running the featurization pipeline.
23. ****Funding Rate Scale Unit Inconsistency (Percentage vs Decimal)****
- **"Failure Mode":** Funding rate z-scores differed by two orders of magnitude between live and backtest.
 - **"Root Cause":** Coinglass UI displayed percentages ('-1.98%'), JS divided by 100 ('-0.0198'), and Python normalizer divided by 100 again ('-0.000198'), whereas historical parquet files held raw decimal floats.
 - **"Fix Applied":** Standardized all funding rate ingestion to absolute decimal fractions with runtime boundary assertions ('abs(funding) < 0.01').
-

Layer 4: Concurrency, Threading & Operating System Resources

24. ****Windows Multiprocessing Spawn Recursion****
- **"Failure Mode":** Model retraining triggered infinite child process spawns, exhausting RAM and locking MT5/CDP ports.
 - **"Root Cause":** On Windows, multiprocessing uses 'spawn', re-executing top-level script imports including engine initialization.

```
- "Fix Applied": Protected all entry points behind if __name__ == '__main__': and factored worker execution into standalone scripts.

25. "Openpyxl File Descriptor Leaks on Disk Cache Seeding"
- "Failure Mode": Python process locked workbook files, crashing subsequent seeding runs on Windows.
- "Root Cause": openpyxl.load_workbook() was executed without closing file handles.
- "Fix Applied": Wrapped workbook operations in explicit 'try-finally' and context-managed blocks.

26. "Thread Lock Contention Blocking WebSocket Event Loop"
- "Failure Mode": High WebSocket tick latency and socket disconnection timeouts.
- "Root Cause": 'live_unified_predictor.py' executed synchronous model reloading and 'time.sleep(0.1)' inside 'with self._lock:', blocking async tick ingest threads.
- "Fix Applied": Offloaded model reloading and heavy DataFrame transformations to asynchronous executors outside the critical tick lock.

---

### Layer 5: Broker Execution & Risk Management

27. "The Orphan Maker (Invalid Emergency Exit Parameter)"
- "Failure Mode": Positions remained open indefinitely during market crash conditions.
- "Root Cause": 'EMERGENCY_HALT' routines passed internal numeric ticket strings to 'BinanceBrokerAdapter.close_position()', which required a Binance pair string ('BTCUSD'). Binance rejected the order with an invalid symbol error.
- "Fix Applied": Enforced symbol-keyed position dispatch with ticker translation across all broker adapters.

28. "Drawdown Baseline Reset and Zeno Position Inversion"
- "Failure Mode": False emergency halts at midnight and negative order units dispatched to exchange.
- "Root Cause": 'daily_art_capital' incorporated unrealized floating PnL at midnight, interpreting normal retracements as daily drawdowns. Furthermore, '(MAX_DD_LIMIT - current_dd)' produced negative multipliers when total drawdown exceeded the configured threshold.
- "Fix Applied": Anchored daily baseline strictly to realized cash balance and clamped sizing multipliers with 'max(0.0, ...)'.

---

## Discussion & Review Questions for Codex

1. "DOM Scraper vs. Direct WebSockets": Given the failure modes uncovered in DOM scraping (zero-guard traps, focus collisions, UI formatting drifts), is continued DOM scraping justified, or should the entire data pipeline migrate to direct WebSocket/REST endpoints (e.g. Binance Futures WS + CoinGlass Enterprise API)?
2. "Feature Parity Governance": What automated continuous integration test patterns should be established to ensure feature calculations in training scripts, backtesters, and live streaming engines cannot diverge silently?
3. "Execution Geometry": In a high-volatility 15m crypto regime, does trailing-stop execution outperform fixed R-R targets when factoring in exchange fees, slippage, and spread rejections?
4. "State Machine Resilience": What architectural patterns best isolate Playwright browser lifecycles from async asyncio trading event loops to prevent UI halts from degrading trade management?
```

Based on the pasted code + the Audit_1 brief, I would **not move Engine 1 beyond testnet yet**. The audit is directionally excellent, but the current code still contains several **P0 live-trading blockers**, and a few "fix applied" items in the brief are only partially implemented or have regressed.

Below is a Codex-style peer review.



1. Executive verdict

Engine 1 has four main remaining risk clusters:

1. Execution parity is still broken

The live broker places a hard Binance `TAKE_PROFIT_MARKET` at 5R, while the live tracker/backtest comments say the strategy should *not* hard-exit at TP and should trail after 5R. This alone can invalidate OOS/live parity.

2. Broker sizing and local risk state diverge

The tracker computes capped units, but the broker adapter recomputes risk independently and ignores the tracker's notional cap and `risk_mult`.

3. State reconciliation is not robust enough for Binance Futures

Binance positions are symbol/side based, but the tracker still reasons heavily through internal `order_id/ticket` concepts. SL/TP hits on exchange can be logged locally with wrong exit price/PnL.

4. Several audit fixes are not actually present in the current code

Examples: `taker-count` positional parsing still exists, `daily_start_capital` still rolls over using unrealized PnL, `openpyxl` workbook handles are still not closed in `load_history_from_disk`, and the TP/trailing mismatch remains.

2. P0 issues I would fix before any live deployment

P0-1 — Conflicting `BINANCE_LIVE` defaults

In `main()`:

```
binance_live = os.environ.get("BINANCE_LIVE", os.environ.get("BINANCE_LIVE", "0")) == "0"
```



Default is effectively dry-run.

But in `LiveTradeTracker.__init__()`:

```
binance_live = os.environ.get("BINANCE_LIVE", "1") != "0"
```

Default is live/testnet execution.

That means if `BINANCE_LIVE` is unset, the startup banner can say no real orders will be sent, while the broker may initialize as non-dry-run.

Fix: make the global default fail-safe everywhere:

```
binance_live = os.environ.get("BINANCE_LIVE", "0") == "1"
```

and use one shared config object.

P0-2 — Tracker notional cap is ignored by broker

Inside `LiveTradeTracker.trigger_entry()`:

```
if notional > MAX_NOTIONAL:
    units = MAX_NOTIONAL / entry_price
```

But then execution goes through:



```
broker_res = self.broker.execute_trade(symbol, direction, entry_price, sl, tp, strategy)
```

And `BinanceBrokerAdapter.execute_trade()` recomputes risk:

```
risk_capital = self.tracker.current_capital * ENGINE_RISK_PCT
```

Then `BinanceBroker.execute_trade()` computes quantity again:

```
qty = self._format_qty(binance_symbol, risk_capital / stop_dist)
```

So the local units cap and `risk_mult` are not guaranteed to reach Binance.

Fix: pass the final approved quantity or final approved `risk_capital` into the broker. Do not let the broker recompute sizing.

Recommended contract:

```
broker.execute_trade(
    symbol=symbol,
    direction=direction,
    entry_price=entry_price,
    sl=sl,
    tp=tp,
    strategy=strategy,
    quantity=units,
)
```

Then the broker only rounds down to exchange step size.

P0-3 — Live broker hard-TP contradicts trailing-only strategy

In `BinanceBroker.execute_trade()`:

```
self._place_algo_conditional(
    binance_symbol,
    opposite_side,
    "TAKE_PROFIT_MARKET",
    final_tp,
    "tp"
)
```

But in `LiveTradeTracker.check_exits()`:

```
# NOTE: No hard TP exit - OOS backtest relies solely on
# trailing stop ratchet after 5R profit.
```

This is a direct live/OOS parity break.

Choose one:

Option A — Hard TP strategy

Keep Binance TP, but update the backtest/live tracker to exit at fixed 5R.

Option B — Trailing-runner strategy

Remove Binance hard TP. Keep only catastrophic SL on exchange, and have the engine update SL after 5R activation.

For safety, I'd use:

- exchange-native catastrophic SL always;
- optional partial TP using fixed quantity, not `closePosition=true`;
- trailing remainder managed by engine.

But the current code cannot claim trailing-only parity while placing full-position `TAKE_PROFIT_MARKET`.

P0-4 — Local PnL/exits use signal entry, not actual execution entry

After broker execution, the tracker stores:

```
self.active_trades[trade_id]["exec_entry"] = broker_res.get("exec_entry")
self.active_trades[trade_id]["exec_sl"] = broker_res.get("exec_sl")
self.active_trades[trade_id]["exec_tp"] = broker_res.get("exec_tp")
```

But live PnL and exit checks still use:

```
entry_price = trade['entry_price']
sl = trade['sl']
tp = trade['tp']
```

Those are the signal prices, not the actual Binance `exec` fill and actual broker SL/TP.

Fix: once the broker confirms a fill, promote execution prices to canonical tracking prices:

```
trade["entry_price"] = broker_res["exec_entry"]
trade["sl"] = broker_res["exec_sl"]
trade["tp"] = broker_res["exec_tp"]
trade["units"] = broker_res["lot"]
```

If you want to preserve model-intended levels, store them separately as:

```
trade["signal_entry"]
trade["signal_sl"]
trade["signal_tp"]
```

P0-5 — `_translate_to_binance_price()` is called but not defined

In `LiveTradeTracker.check_exits()`:

```
exec_sl = self._translate_to_binance_price(t, t["sl"])
```

But `LiveTradeTracker` has no `_translate_to_binance_price()` method in the pasted code.

That can break SL heartbeat modification.

Fix: either implement it or remove it. Since symbols are already Binance symbols, the simplest version is probably just broker-side formatting:


```
def _translate_to_binance_price(self, trade: dict, price: float) -> float:
    return float(price)
```

Better: delegate to broker formatting if available.

P0-6 — Order slicing can create phantom filled quantity

In `BinanceBroker.execute_trade()`, for later market slices:

```
mkt_result = self._request(...)
if mkt_result and "orderId" in mkt_result:
    all_order_ids.append(int(mkt_result["orderId"]))
else:
    log.error(...)

total_filled_qty += slice_qty
```

`total_filled_qty` is incremented even if the market order failed.

That can cause the engine to believe more quantity is open than actually filled.

Fix: increment only after confirmed execution. Ideally parse `executedQty`.

P0-7 — Average execution price is wrong for sliced/mixed orders

Current average price is based on only `entry_result`, which is often the last successful order or first GTX order:

```
cum_quote = float(entry_result.get("cumQuote", 0.0))
exec_qty = float(entry_result.get("executedQty", 0.0))
avg_price = cum_quote / exec_qty
```

For multiple slices, this is not a weighted average across all fills.

Fix: aggregate every fill.

```
avg_price = sum(fill_qty_i * fill_price_i) / sum(fill_qty_i)
```

If Binance response does not provide reliable fill details, query user trades by `orderId`.

P0-8 — Rounding can violate exchange step/tick sizes

`Current_round_step()`:

```
precision = int(round(-math.log10(step))) if step < 1 else 0
factor = 10 ** precision
```

This fails for valid non-power-of-ten increments like 0.005, 0.25, etc.

Fix: round by division against the step size, ideally with `Decimal`.

Also, quantities should usually be rounded **down**, not nearest, to avoid over-risking.

P0-9 — self.history is not initialized if no log file exists

In `LiveTradeTracker.__init__()` you initialize:

```
self.closed_trades: List[dict] = []
```

But many methods use:

```
self.history
```

`load_history()` returns early if no file exists, without setting `self.history`.

Fix:

```
self.history: List[dict] = []
```

before calling `load_history()`.

P0-10 — Daily rollover still uses unrealized PnL

The audit says daily baseline was anchored to realized balance, but current code still does:

```
unrealized_pnl = sum(t.get('live_pnl_usd', 0.0) for t in active_list)
current_equity = self.current_capital + unrealized_pnl
self.daily_start_capital = current_equity
```

That can still create midnight drawdown artifacts.

Fix: use realized cash/equity from broker wallet balance, or at minimum `self.current_capital` only.

P0-11 — Broker reconciliation is not symbol-native enough

Binance Futures positions are not MT5-style tickets. They are effectively keyed by:

```
symbol + positionSide
```

But the tracker still maps position existence through `order_id`.

This is fragile:

```
broker_positions[int(p.ticket)] = p
```

where `p.ticket` is actually derived from local active trades.

Fix: reconcile by symbol.

```
local active symbols
vs
Binance positionRisk positionAmt by symbol
```

Then for closed positions, fetch realized PnL from Binance user trades/income history.

Your current `get_position_history_profit()` returns:

```
return 0.0, 0.0
```

So broker-side SL/TP closures cannot be logged accurately.

P0-12 — `modify_sltp()` creates a naked window

Current logic cancels old SL/TP first:

```
# Cancel ALL existing algo orders first
...
new_sl_res = self._place_algo_conditional(...)
```

If the new SL placement fails, the system emergency-closes, but there is still a window where the position has no protective stop.

Better pattern:

1. Place replacement protective SL first if exchange permits.
2. Confirm accepted.
3. Cancel stale protection.
4. Place/refresh TP/trailing orders.

If Binance rejects multiple `closePosition=true` stops, use reduce-only quantity stops rather than global close-position orders, or implement a two-stage emergency guard.

3. Audit claims that look only partially implemented

These are worth reconciling before handing to Codex.

Taker count fix not actually present

Audit says structural regex parsing was added, but current JS still does positional extraction:

```
if (numVals.length >= 3) {
  data.taker_buy_count = numVals[numVals.length - 3];
  data.taker_sell_count = numVals[numVals.length - 2];
}
```

So the original [14, 520, 488] bug can still occur.

Funding normalization is still risky

JS does:

```
data.funding_rate = !isNaN(row) ? (row * 100).toFixed(4) + '%' : t;
```

Then Python `parse_float()` strips % but does not divide by 100.

Later `featurize()` does:

```
fr = fr.apply(lambda v: v / 100.0 if abs(v) >= 0.005 else v)
```

This can work for some display formats, but it is brittle. Funding should be normalized once at ingestion with strict assertions:

```
expected live funding decimal: abs(funding) < 0.01
```

Anything outside that should be rejected or explicitly converted.

Openpyxl handle leak still exists

`LiveSixStrategyPredictor.load_history_from_disk()`:

```
wb = openpyxl.load_workbook(combined_path, read_only=True)
...
```

No `wb.close()`.

Use:

```
try:
  wb = openpyxl.load_workbook(..., read_only=True)
  ...
finally:
  wb.close()
```

Stop-loss floor parity still exists

Audit says hardcoded SL floor overrides were removed, but live code still has:

```
MIN_STOP_PCT = {...}
```

and live predictor also floors ATR:

```
atr_val = max(atr_val, snap.price * 0.0005)
```

This may be sensible for execution safety, but then the backtest must include the same floor. Otherwise OOS/live parity remains broken.

VWAP recross exit is not present in active six-strategy engine

The audit says VWAP recross exits were added for S4/S5/S6, but the actual active

`LiveSixStrategyPredictor` does not appear to implement a VWAP-touch exit listener. The older

`UnifiedLivePredictor` has VWAP fields, but `main()` uses:

```
from six_strategy_engine import LiveSixStrategyPredictor
```

So this fix may not be active.

4. Answers to the four Codex review questions

Q1 — DOM scraper vs direct WebSockets/API?

For live trading, DOM scraping should not be a primary data source.

Use DOM only as:

- visual diagnostics;
- fallback;
- shadow comparison;

- manual observability.

Critical features should come from deterministic APIs/streams.

For crypto Binance Futures, most required fields can be sourced directly:

Feature	Better source
Price	Binance WS aggTrade, bookTicker, markPrice, kline
CVD	Compute from agg trades / taker side
Funding	Binance premium/funding endpoints
OI	Binance open interest endpoints
Liquidations	Binance force-order/liquidation stream
Bid/ask	Binance depth/bookTicker streams
RSI	compute locally
Taker buy/sell volume/count	compute locally from trades
VWAP	compute locally
POC	compute from agg trades/depth, not klines

If the alpha specifically depends on **CoinGlass aggregated cross-exchange CVD/OI/liquidations**, then use CoinGlass' official API or enterprise feed. DOM legends are too fragile for order-triggering.

My recommendation:

```
Phase 1: Keep DOM, but mark it non-authoritative.
Phase 2: Build direct Binance feature pipeline.
Phase 3: Run DOM and direct feed in parallel, compare deltas.
Phase 4: Remove DOM from trade gating.
```

Q2 — Feature parity governance

You need a single shared feature package imported by:

```
trainer
backtester
live predictor
replay simulator
```

Right now you have multiple `featurize()` implementations and multiple strategy engines. That is the root of many parity bugs.

Minimum CI gates:

1. Golden fixture tests

Store a small canonical dataset:

```
tests/fixtures/BTCUSD_15m_1000bars.parquet
```

Then assert that training and live feature code produce identical values.

Example invariants:

```
atr_live == atr_train
zc20_live == zc20_train
mc_live == mc_train
rsi_live == rsi_train
funding_unit_live == funding_unit_train
```

2. Future-leakage test

Perturb future rows and assert earlier features do not change.

```
features[0:t] must be identical even if raw[t+1:] changes
```

3. Schema/unit contracts

Every feature should have:

```
{
  "name": "funding",
  "unit": "decimal fraction",
  "valid_min": -0.01,
  "valid_max": 0.01
}
```

Reject values outside expected ranges.

4. Model manifest hash

Every model artifact should include:

```
{
  "git_sha": "...",
  "feature_code_hash": "...",
  "feature_columns": [...],
  "strategy_code_hash": "...",
  "training_data_cutoff": "...",
  "threshold": 0.62,
  "label_geometry": {
    "tp_mult": 5.0,
    "trail_atr": 0.8,
    "sl_mult": 1.0,
    "maxBars": 288
  }
}
```

The live engine should refuse to load models if the current feature code hash does not match.

5. Replay-mode integration test

Record raw live packets for one day, then replay them through:

```
live pipeline -> features -> signals -> orders
```

Compare outputs against offline backtest on the same bars.

Q3 — Trailing stop vs fixed R-R target?

The answer depends on the distribution of trade returns, but your current system must pick one and make live/backtest identical.

For 15m crypto:

Fixed TP advantages

- Simpler execution.
- Easier exchange-native protection.
- More stable win-rate.
- Lower risk of giving back open profit.
- Easier to model fees/slippage.

Trailing-runner advantages

- Captures fat-tail crypto moves.
- Better for trend-pullback systems.
- Can improve expectancy even with lower win rate.
- More consistent with 5R activation + 0.8ATR trail logic.

Given your stated OOS logic, the intended system appears to be:

No hard TP.
Activate trailing after 5R.
Exit on trailing stop or timeout.



Copy

So the current Binance hard TP should be removed or converted to partial TP.

Best practical hybrid:

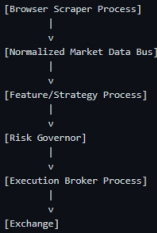
1. Initial exchange SL immediately after entry.
2. Optional 30-50% reduce-only TP at 4R-5R.
3. Trail remaining size with exchange SL modifications.
4. No full-position hard TP unless backtest includes it.

Copy

But if you cannot reliably modify stops under latency, then fixed TP may be safer operationally.

Q4 — State machine resilience / Playwright isolation

The clean architecture should be process-isolated:



Copy

The Playwright process should never directly influence order management. It should publish normalized observations only.

Recommended components:

Browser actor

Responsibilities:

- maintain Chromium;
- scrape DOM/API intercepts;
- emit IndicatorSnapshot;
- self-restart if unhealthy.

No trading logic.

Market-data actor

Responsibilities:

- Binance WS/REST ingestion;
- candle building;
- direct feature source computation;
- timestamp normalization.

Strategy actor

Responsibilities:

- consume closed candles only;
- compute features;
- evaluate strategies;
- output SignalIntent.

Risk actor

Responsibilities:

- portfolio limits;
- cooldowns;
- duplicate prevention;
- max notional;
- final sizing.

Broker actor

Responsibilities:

- submit orders;
- reconcile fills;
- maintain position state from Binance;
- manage protective orders.

Journal

Every state transition should be append-only:

SIGNAL_CREATED
RISK_ACCEPTED
ORDER_SUBMITTED
ORDER_FILLED
SL_PLACED
TP_PLACED
POSITION_RECONCILED
POSITION_CLOSED



Copy

On restart, rebuild state from journal + Binance positionRisk.

5. Concrete priority patch queue for Codex

If I were giving Codex implementation instructions, I would do this order:

Patch 1 — Fail-safe live config

- One EngineSettings object.
- BINANCE_LIVE default 0.
- BINANCE_USE_TESTNET default true.
- Startup should print exact mode from the same object used by broker.

Patch 2 — Broker sizing contract

- Tracker computes final quantity.
- Broker accepts final quantity.
- Broker no longer recomputes risk.
- Broker rounds quantity down.
- Reject if rounded quantity < min qty or min notional.

Patch 3 — Execution parity



Choose one:

- remove Binance TP and use trailing-only; or
- add hard TP to backtest/live tracker.

Do not leave both inconsistent.

Patch 4 — Canonical execution state

After fill:

```
trade["entry_price"] = exec_entry
trade["sl"] = exec_sl
trade["tp"] = exec_tp
trade["units"] = exec_lot
```

Copy

Keep signal prices separately.

Patch 5 — Binance reconciliation rewrite

Reconcile by:

```
symbol
positionAmt
entryPrice
unRealizedProfit
```

Copy

not local order ticket.



Fetch realized PnL from Binance income/userTrades when position disappears.

Patch 6 — Stop modification safety

- Do not cancel old SL before confirmed new SL unless unavoidable.
- If unavoidable, emergency close must be synchronous and verified.
- Add explicit needs_manual_attention on any protective-order failure.

Patch 7 — Shared feature library

Delete duplicate featurize() implementations or make them wrappers around one canonical module.

Patch 8 — DOM demotion

Make DOM features non-authoritative until they pass:

```
freshness
schema
range
cross-source sanity
```

Copy

For trade gating, prefer direct Binance/API features.

6. Bottom-line recommendation

The system is close in concept, but I would classify current Engine 1 as:

```
Research/testnet-ready: yes
Small live capital ready: not yet
Production-ready: no
```

Copy

The biggest immediate fixes are:

1. unity live/dry-run config;
2. fix broker sizing and rounding;
3. remove the TP/trailing parity contradiction;
4. reconcile positions by Binance symbol/side, not ticket;
5. promote actual execution prices into canonical local trade state;
6. move feature engineering into one shared tested module.

Once those are done, the remaining DOM/data issues become manageable rather than existential.

📄 🍌 💬 ...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

📄 🍌 💬 ...

AI Chat

The AI Costco — one membership →

Ask anything, create anything

+ GPT-5.5 Pro

